

Python Features Reference for Scientific IO & Geometry Code

A consolidated guide to language features useful for pre/post-FEM pipelines, geometry assembly, and TetGen/Fortran IO. Organised by category, not importance — all of these are worth knowing.

1. `dataclasses` — Your Primary Building Block

Basic usage

```
from dataclasses import dataclass, field

@dataclass
class Material:
    name: str
    vp: float      # P-wave velocity m/s
    vs: float      # S-wave velocity m/s
    density: float # kg/m³
    material_id: int
```

Auto-generates `__init__`, `__repr__`, `__eq__`. No boilerplate.

`field()` — control individual fields

```
@dataclass
class ReceiverArray:
    positions: list = field(default_factory=list) # mutable default – never use
    []
    label: str      = field(default="rx")
    _cache: dict    = field(default_factory=dict, init=False, repr=False)
    #               ^^^^^^^^^^^  ^^^^^^^^^^^
    #               not in __init__ not in __repr__
```

`init=False` fields are set only in `__post_init__`, never by the caller. `repr=False` hides internal/cache fields from debug output.

`__post_init__` — validation at construction time

```
@dataclass
class EarthLayer:
    depth_top: float
    thickness: float
    material: Material
```

```
def __post_init__(self):
    if self.thickness <= 0:
        raise ValueError(f"thickness must be positive, got {self.thickness}")
    if self.depth_top < 0:
        raise ValueError(f"depth_top must be >= 0, got {self.depth_top}")
```

Runs automatically after `__init__`. The right place for all validation, unit conversion, and derived-field initialisation.

`frozen=True` — immutable value types

```
@dataclass(frozen=True)
class Vec3:
    x: float
    y: float
    z: float

    def __add__(self, other: "Vec3") -> "Vec3":
        return Vec3(self.x + other.x, self.y + other.y, self.z + other.z)

    def norm(self) -> float:
        return (self.x**2 + self.y**2 + self.z**2) ** 0.5
```

Frozen dataclasses are hashable, can be used as dict keys, and protect you from accidentally mutating a coordinate mid-assembly.

`slots=True` (Python 3.10+) — memory efficiency

```
@dataclass(slots=True)
class Node:
    x: float
    y: float
    z: float
    index: int = -1
```

With `slots=True`, instances use `__slots__` instead of `__dict__`. Meaningful when you have tens of thousands of nodes — roughly 30–50% less memory and slightly faster attribute access.

`KW_ONLY` sentinel — force keyword arguments

```
from dataclasses import dataclass, KW_ONLY

@dataclass
class BoxAnomaly:
    x_min: float
```

```

x_max: float
_: KW_ONLY          # everything after this is keyword-only
material: Material
label: str = "anomaly"

# BoxAnomaly(0, 100, material=granite)  ✓
# BoxAnomaly(0, 100, granite)          ✗ - clear error

```

Prevents positional argument confusion for objects with many similar-typed fields.

2. `@property` — Derived Quantities as Attributes

```

@dataclass
class EarthLayer:
    depth_top: float
    thickness: float
    material: Material

    @property
    def depth_bottom(self) -> float:
        return self.depth_top + self.thickness

    @property
    def poisson_ratio(self) -> float:
        r = self.material.vp / self.material.vs
        return (r**2 - 2) / (2 * (r**2 - 1))

    @property
    def impedance(self) -> float:
        return self.material.density * self.material.vp

```

Callers write `layer.poisson_ratio`, not `layer.compute_poisson_ratio()`. The value is always consistent because it is always derived — you cannot store an inconsistent one.

Setter for coercion

```

@dataclass
class IOConfig:
    _project_dir: Path = field(init=False)
    project_dir_input: str = "" # raw input

    @property
    def project_dir(self) -> Path:
        return self._project_dir

    @project_dir.setter
    def project_dir(self, value):
        self._project_dir = Path(value) # accept str or Path

```

3. @classmethod Factory Methods — Named Constructors

The most important pattern for objects with multiple natural construction modes.

```
@dataclass
class ReceiverArray:
    positions: list[Vec3]
    component: str = "all"

    @classmethod
    def from_line(cls, start: Vec3, end: Vec3, n: int, **kwargs) ->
"ReceiverArray":
    """Evenly-spaced linear array."""
    t_vals = [i / (n - 1) for i in range(n)]
    positions = [
        Vec3(
            start.x + t * (end.x - start.x),
            start.y + t * (end.y - start.y),
            start.z + t * (end.z - start.z),
        )
        for t in t_vals
    ]
    return cls(positions=positions, **kwargs)

    @classmethod
    def from_grid(cls, origin: Vec3, dx: float, dy: float,
                  nx: int, ny: int, **kwargs) -> "ReceiverArray":
    """Regular 2D surface grid."""
    positions = [
        Vec3(origin.x + i * dx, origin.y + j * dy, origin.z)
        for i in range(nx)
        for j in range(ny)
    ]
    return cls(positions=positions, **kwargs)

    @classmethod
    def from_csv(cls, path: Path, **kwargs) -> "ReceiverArray":
    import csv
    with open(path) as f:
        positions = [Vec3(*map(float, row)) for row in csv.reader(f)]
    return cls(positions=positions, **kwargs)
```

Each factory is self-documenting. `**kwargs` passes through to `__init__` so you never have to duplicate keyword arguments.

4. Protocol — Structural Interfaces Without Inheritance

Python's alternative to abstract base classes. Any class that implements the right methods is automatically compatible — no `class Foo(MyABC)` needed.

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class GeometryContributor(Protocol):
    """Anything that can contribute geometry to the mesh assembly."""
    def get_nodes(self) -> list["Node"]: ...
    def get_facets(self) -> list["Facet"]: ...
    def get_regions(self) -> list["Region"]: ...
    def get_holes(self) -> list["Hole"]: ...

# EarthLayer, BoxAnomaly, PMLBlock all satisfy this
# without inheriting from anything – duck typing + type safety

def assemble(contributors: list[GeometryContributor]) -> "PolyFile":
    ...

# Runtime check:
assert isinstance(my_layer, GeometryContributor) # works with @runtime_checkable
```

5. `enum` — Replace Magic Integers and Strings

`IntEnum` for TetGen attributes and boundary markers

```
from enum import IntEnum, auto

class MaterialID(IntEnum):
    BACKGROUND = 1
    LAYER_2    = 2
    ANOMALY    = 10
    PML        = 99

class BoundaryMarker(IntEnum):
    FREE_SURFACE = 1
    PML_INTERFACE = 2
    PML_OUTER    = 3
```

`IntEnum` values compare equal to plain `int`, so they write directly to files:

```
f.write(f"{region.attribute}\n") # works even if attribute is a MaterialID
```

`StrEnum` (Python 3.11+) for wavelet / solver names

```

from enum import StrEnum

class Wavelet(StrEnum):
    RICKER    = "ricker"
    GAUSSIAN  = "gaussian"
    SINE      = "sine"

# Compares to plain strings, serializes cleanly
source = Source(wavelet=Wavelet.RICKER)
f.write(source.wavelet)    # writes "ricker"

```

Flag for bitfield options

```

from enum import Flag, auto

class OutputComponent(Flag):
    VX = auto()
    VY = auto()
    VZ = auto()
    P  = auto()
    ALL = VX | VY | VZ | P

cfg = OutputConfig(components=OutputComponent.VX | OutputComponent.VZ)
if OutputComponent.VX in cfg.components:
    ...

```

6. `pathlib.Path` — File Paths as Objects

Never use raw strings for paths.

```

from pathlib import Path

@dataclass
class IOConfig:
    project_dir: Path
    run_name: str

    def __post_init__(self):
        self.project_dir = Path(self.project_dir)    # coerce str → Path

    @property
    def input_dir(self) -> Path:
        return self.project_dir / "inputs" / self.run_name

    @property
    def output_dir(self) -> Path:
        return self.project_dir / "outputs" / self.run_name

```

```

@property
def poly_file(self) -> Path:
    return self.input_dir / f"{self.run_name}.poly"

@property
def node_file(self) -> Path:
    return self.output_dir / f"{self.run_name}.1.node"

def ensure_dirs(self) -> None:
    self.input_dir.mkdir(parents=True, exist_ok=True)
    self.output_dir.mkdir(parents=True, exist_ok=True)

```

Key methods you'll use: `.stem`, `.suffix`, `.exists()`, `.glob("*.node")`, `.with_suffix(".ele")`, `.parent`, `.resolve()`.

7. Type Hints — Readable and Tool-Friendly

Basic annotations

```

def write_nodes(nodes: list[Node], path: Path) -> int:
    """Returns number of nodes written."""
    ...

```

Optional vs X | None (3.10+)

```

# Old style
from typing import Optional
def find_layer(depth: float) -> Optional[EarthLayer]: ...

# New style (3.10+) – preferred
def find_layer(depth: float) -> EarthLayer | None: ...

```

TypeAlias for domain-specific types

```

from typing import TypeAlias

NodeIndex: TypeAlias = int
MaterialAttribute: TypeAlias = int
Coordinate: TypeAlias = tuple[float, float, float]

```

ClassVar — class-level constants that aren't instance fields

```

from dataclasses import dataclass
from typing import ClassVar

@dataclass
class Source:
    frequency: float
    wavelet: str = "ricker"

    VALID_WAVELETS: ClassVar[set[str]] = {"ricker", "gaussian", "sine"}
    # ^ not included in __init__ or __repr__ – it's class metadata

    def __post_init__(self):
        if self.wavelet not in self.VALID_WAVELETS:
            raise ValueError(f"Invalid wavelet: {self.wavelet!r}")

```

8. `__repr__` and `__str__` — Debugging That Helps You

Dataclasses give a decent `__repr__` for free. Override `__str__` for human-readable summaries you'd want in logs or reports.

```

@dataclass
class EarthLayer:
    depth_top: float
    thickness: float
    material: Material

    def __str__(self) -> str:
        return (f"EarthLayer [{self.depth_top}-{self.depth_bottom} m"
                f" | {self.material.name}]")

    # __repr__ is auto-generated by @dataclass – leave it alone
    # It gives: EarthLayer(depth_top=0.0, thickness=50.0, material=Material(...))

```

Use `!r` in format strings to call `__repr__` on nested objects:

```

raise ValueError(f"Invalid material: {self.material!r}")

```

9. `__add__` and Operator Overloading — Geometry Assembly

```

@dataclass
class PolyFile:
    nodes: list[Node]
    facets: list[Facet]
    holes: list[Hole]

```



```

regions: list[Region]

def __add__(self, other: "PolyFile") -> "PolyFile":
    """Merge two PolyFiles, re-indexing the second."""
    offset = len(self.nodes)
    shifted = [
        Facet([i + offset for i in f.node_indices], f.boundary_marker)
        for f in other.facets
    ]
    return PolyFile(
        nodes = self.nodes + other.nodes,
        facets = self.facets + shifted,
        holes = self.holes + other.holes,
        regions = self.regions + other.regions,
    )

def __len__(self) -> int:
    return len(self.nodes)

def __bool__(self) -> bool:
    return len(self.nodes) > 0

```

Then: `full_mesh = domain_poly + pml_poly` — reads like what it is.

10. Method Chaining — Builder Pattern

Return `self` from mutating methods to enable chaining. Used in Ceres Solver, Gmsh Python API, SQLAlchemy.

```

class ModelAssembler:
    def __init__(self, domain):
        self.domain = domain
        self._layers = []
        self._anomalies = []
        self._pml = None

    def add_layer(self, layer: EarthLayer) -> "ModelAssembler":
        self._layers.append(layer)
        return self

    def add_anomaly(self, anomaly: BoxAnomaly) -> "ModelAssembler":
        self._anomalies.append(anomaly)
        return self

    def set_pml(self, pml: PMLBlock) -> "ModelAssembler":
        self._pml = pml
        return self

    def build(self) -> PolyFile:
        ...

```

```
# Usage reads like a specification:
poly = (
    ModelAssembler(domain)
    .add_layer(EarthLayer(0, 50, sand))
    .add_layer(EarthLayer(50, 150, clay))
    .add_anomaly(BoxAnomaly(150, 250, 150, 250, -80, -120, granite))
    .set_pml(PMLBlock.from_domain(domain, thickness=30, material=pml_mat))
    .build()
)
```

11. `contextlib` — Clean Resource Management

`contextmanager` for file writing sections

```
from contextlib import contextmanager

@contextmanager
def section(f, title: str):
    """Write a labelled comment block around a .poly file section."""
    f.write(f"# --- {title} ---\n")
    yield f
    f.write("\n")

# Usage:
with open(path, "w") as f:
    with section(f, "Node list"):
        f.write(f"{len(nodes)} 3 0 0\n")
        for i, node in enumerate(nodes, 1):
            f.write(f"{i} {node.x:.6f} {node.y:.6f} {node.z:.6f}\n")
```

`suppress` for optional teardown

```
from contextlib import suppress

with suppress(FileNotFoundError):
    old_output.unlink() # delete if exists, ignore if not
```

12. `textwrap` — Clean Multi-line String Formatting

```
import textwrap
from datetime import datetime

def _file_header(self) -> str:
    return textwrap.dedent(f"""\n
```

```

    # TetGen .poly file
    # Generated : {datetime.now().isoformat(timespec='seconds')}
    # Model      : {self.model_name}
    # Layers     : {self.n_layers}
    # PML        : {self.pml_thickness} m
    # Units      : metres
    """)

```

`dedent` strips leading whitespace so the string in code can be indented naturally without that indentation appearing in the file. The `\` after `"""` avoids a leading blank line.

13. `tomllib` / `tomli` — Config File Round-Trip

Built-in since Python 3.11. For 3.9/3.10 use `pip install tomli`.

```

import tomllib    # read-only, stdlib 3.11+
import tomli_w    # write, third-party: pip install tomli-w

# Serialise a survey to TOML
def to_dict(self) -> dict:
    return {
        "source": {
            "x": self.source.x,
            "y": self.source.y,
            "z": self.source.z,
            "frequency": self.source.frequency,
            "wavelet": str(self.source.wavelet),
        },
        "layers": [
            {"depth_top": l.depth_top,
             "thickness": l.thickness,
             "material": l.material.name}
            for l in self.layers
        ],
    }

def save_toml(self, path: Path) -> None:
    import tomli_w
    with open(path, "wb") as f:
        tomli_w.dump(self.to_dict(), f)

@classmethod
def load_toml(cls, path: Path) -> "Survey":
    with open(path, "rb") as f:
        data = tomllib.load(f)
    return cls.from_dict(data)

```

This gives you a human-readable, version-controllable record of every run.

14. `dataclasses.asdict` / `astuple` — Free Serialisation

```
from dataclasses import asdict, astuple

layer = EarthLayer(depth_top=0, thickness=50, material=sand)

asdict(layer)
# {'depth_top': 0, 'thickness': 50,
#  'material': {'name': 'sand', 'vp': 1800, ...}}

# Round-trip
layer2 = EarthLayer(**{k: v for k, v in asdict(layer).items()
                       if k != 'material'}, material=sand)
```

`asdict` recurses into nested dataclasses automatically. Useful for JSON/TOML export without writing `to_dict` manually for simple cases.

15. `__slots__` Without `dataclasses` — For Hot Inner Loop Classes

For pure geometry primitives used in tight loops, hand-written `__slots__` classes can be faster than dataclasses:

```
class Node:
    __slots__ = ("x", "y", "z", "index")

    def __init__(self, x: float, y: float, z: float, index: int = -1):
        self.x = x; self.y = y; self.z = z; self.index = index

    def __repr__(self):
        return f"Node({self.x}, {self.y}, {self.z})"
```

For most code, `@dataclass(slots=True)` (3.10+) gives you both.

16. `collections.namedtuple` vs `dataclass(frozen=True)`

```
from collections import namedtuple

# namedtuple – immutable, lightweight, tuple-compatible
BBox = namedtuple("BBox", ["x_min", "x_max", "y_min", "y_max", "z_min", "z_max"])

# frozen dataclass – immutable, but supports methods and type hints
@dataclass(frozen=True)
class BBox:
    x_min: float; x_max: float
    y_min: float; y_max: float
```

```

z_min: float; z_max: float

@property
def volume(self) -> float:
    return ((self.x_max - self.x_min) *
            (self.y_max - self.y_min) *
            (self.z_max - self.z_min))

def contains(self, p: Vec3) -> bool:
    return (self.x_min <= p.x <= self.x_max and
            self.y_min <= p.y <= self.y_max and
            self.z_min <= p.z <= self.z_max)

```

Use `namedtuple` only for dead-simple value containers with no methods. Prefer `frozen=True` `dataclass` for anything that needs logic.

17. logging — Proper Progress and Debug Output

Never use `print()` in library code. Use `logging`.

```

import logging

log = logging.getLogger(__name__)

class PolyFileWriter:
    def write(self, path: Path) -> Path:
        log.info(f"Writing .poly file → {path}")
        log.debug(f"    {len(self.poly.nodes)} nodes, "
                  f"    {len(self.poly.facets)} facets, "
                  f"    {len(self.poly.regions)} regions")
        ...
        log.info("Done.")
        return path

```

The caller configures the log level — your library stays silent unless asked. In a script:

```
logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
```

18. warnings — Non-Fatal Cautions

```

import warnings

def build(self) -> PolyFile:
    if self._pml is None:
        warnings.warn(

```

```

        "No PML defined. The model has no absorbing boundary. "
        "Reflections from domain edges will contaminate the solution.",
        UserWarning,
        stacklevel=2,    # points warning at the caller, not this function
    )

```

Better than printing — the user can filter, suppress, or promote to error.

19. `__init_subclass__` — Registry Pattern for Geometry Types

Useful if you want plugins or auto-registration of custom geometry contributors:

```

class GeometryBase:
    _registry: ClassVar[dict[str, type]] = {}

    def __init_subclass__(cls, geometry_type: str = "", **kwargs):
        super().__init_subclass__(**kwargs)
        if geometry_type:
            GeometryBase._registry[geometry_type] = cls

class EarthLayer(GeometryBase, geometry_type="layer"):
    ...

class SphereAnomaly(GeometryBase, geometry_type="sphere"):
    ...

# Load from TOML:
# geometry_type = "sphere" → SphereAnomaly(**params)
cls = GeometryBase._registry["sphere"]
obj = cls(**params)

```

20. `numpy` Patterns for Geometry

Vectorised node generation

```

import numpy as np

def box_nodes(x_min, x_max, y_min, y_max, z_min, z_max) -> np.ndarray:
    """Return (8, 3) array of box corner coordinates."""
    return np.array([
        [x_min, y_min, z_min],
        [x_max, y_min, z_min],
        [x_max, y_max, z_min],
        [x_min, y_max, z_min],
        [x_min, y_min, z_max],
        [x_max, y_min, z_max],
        [x_max, y_max, z_max],
        [x_min, y_max, z_max],
    ])

```

```

        [x_min, y_max, z_max],
    ])

# Then wrap into Node objects if needed:
nodes = [Node(*row) for row in box_nodes(...)]

```

`np.loadtxt` / `np.savetxt` for TetGen output files

```

# Read TetGen .node file (skip header)
data = np.loadtxt("mesh.1.node", skiprows=1)
indices = data[:, 0].astype(int)
coords = data[:, 1:4]

# Read .ele file
elems = np.loadtxt("mesh.1.ele", skiprows=1, dtype=int)[: , 1:5]

```

21. f-string Features Worth Knowing

```

x = 3.14159265
n_nodes = 12483

# Width and precision
f"{x:.4f}"          # "3.1416"
f"{x:12.4f}"        # "          3.1416" (right-aligned in 12-wide field)
f"{n_nodes:>10d}"    # "          12483"
f"{n_nodes:<10d}"    # "12483          "

# Thousands separator
f"{n_nodes:,}"      # "12,483"

# = flag for debugging (Python 3.8+)
f"{x:=.3f}"         # "x=3.142"
f"{n_nodes=}"       # "n_nodes=12483"

# !r and !s
f"{layer!r}"        # calls __repr__
f"{layer!s}"        # calls __str__

```

The `=` flag (`f"{x=}"`) is particularly useful for quick debug prints.

22. `match` / `case` — Structural Pattern Matching (Python 3.10+)

Useful for dispatching on geometry type or command input:

```
def describe(contributor) -> str:
    match contributor:
        case EarthLayer(depth_top=d, material=Material(name=name)):
            return f"Layer at {d} m depth, material: {name}"
        case BoxAnomaly(material=Material(name=name)):
            return f"Box anomaly, material: {name}"
        case PMLBlock(thickness=t):
            return f"PML, thickness={t} m"
        case _:
            return f"Unknown: {contributor!r}"
```

Also useful for parsing user input from CLI:

```
match answer.strip().lower():
    case "y" | "yes":
        has_anomaly = True
    case "n" | "no":
        has_anomaly = False
    case _:
        raise ValueError(f"Expected yes/no, got {answer!r}")
```

Quick Reference Summary

Need	Feature
Clean data containers	<code>@dataclass</code>
Validation at construction	<code>__post_init__</code>
Immutable value types	<code>@dataclass(frozen=True)</code>
Memory-efficient instances	<code>@dataclass(slots=True)</code> (3.10+)
Force keyword arguments	<code>KW_ONLY</code> sentinel
Multiple construction modes	<code>@classmethod</code> factory methods
Derived / computed quantities	<code>@property</code>
Decoupled IO without inheritance	<code>Protocol</code>
Replace magic integers/strings	<code>IntEnum</code> , <code>StrEnum</code> , <code>Flag</code>
File paths	<code>pathlib.Path</code>
Readable type annotations	type hints + <code>TypeAlias</code> + <code>ClassVar</code>
Merge geometry objects	<code>__add__</code> operator overload
Fluent assembly API	Method chaining (<code>return self</code>)

Need	Feature
Clean file section writing	<code>contextlib.contextmanager</code>
Multi-line file headers	<code>textwrap.dedent</code>
Config persistence / reproducibility	<code>tomllib</code> + <code>tomli_w</code>
Free dict serialisation	<code>dataclasses.asdict</code>
Progress / debug output	<code>logging</code>
Non-fatal warnings	<code>warnings.warn</code>
Auto-register geometry types	<code>__init_subclass__</code>
Batch node arithmetic	<code>numpy</code> vectorised ops
Dispatch on geometry type	<code>match</code> / <code>case</code> (3.10+)
Quick debug prints	f-string = flag (<code>f"{x=}"</code>)